```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# ---------------------------------------------------------
# 1. SETUP: Create a Mock Animal Dataset (Cats vs Dogs)
# ---------------------------------------------------------
# Let's generate a dataset with 20 features (e.g., weight, ear shape, meow volume, barking frequency, etc.)
X_raw, y_raw = make_classification(
    n_samples=200,
    n_features=20,
    n_informative=8,  # Only 8 features are truly relevant
    n_redundant=4,
    random_state=42
)

# Name the features for realistic animal metrics
feature_names = [
    "Weight", "Height", "Ear_Shape", "Bark_Freq", "Meow_Vol", "Tail_Length",
    "Claw_Retraction", "Playfulness", "Snout_Length", "Purr_Freq", "Fur_Thickness",
    "Whiskers_Length", "Night_Vision", "Sociability", "Aggression", "Speed",
    "Shedding_Level", "Bite_Force", "Water_Love", "Sleep_Hours"
]

X = pd.DataFrame(X_raw, columns=feature_names)
y = pd.Series(y_raw) # 0 for Cat, 1 for Dog

# Split into training and testing sets (Fixed the typo here!)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"Dataset initialized with {X.shape[0]} animals and {X.shape[1]} features.")

# ---------------------------------------------------------
# 2. GENETIC ALGORITHM CONFIGURATION
# ---------------------------------------------------------
POP_SIZE = 20          # Number of chromosomes in the population
GENERATIONS = 15       # Number of iterations
MUTATION_RATE = 0.1    # Probability of flipping a gene
NUM_FEATURES = X.shape[1]

# ---------------------------------------------------------
# 3. CORE GA FUNCTIONS
# ---------------------------------------------------------

def initialize_population(pop_size, num_features):
    """Generates a random initial population of binary chromosomes (1 = keep feature, 0 = drop)."""
    pop = np.random.randint(2, size=(pop_size, num_features))
    # Ensure no chromosome is completely empty (all zeros)
    for i in range(pop_size):
        if np.sum(pop[i]) == 0:
            pop[i, np.random.randint(num_features)] = 1
    return pop

def calculate_fitness(chromosome, X_tr, X_te, y_tr, y_te):
    """Evaluates chromosome fitness based on accuracy and a penalty for keeping too many features."""
    selected_indices = np.where(chromosome == 1)[0]

    # If no features selected, it gets worst possible fitness
    if len(selected_indices) == 0:
        return 0.0

    # Filter dataset to selected features
    X_tr_sub = X_tr.iloc[:, selected_indices]
    X_te_sub = X_te.iloc[:, selected_indices]

    # Train a quick classifier
    clf = RandomForestClassifier(n_estimators=50, random_state=42)
    clf.fit(X_tr_sub, y_tr)
    preds = clf.predict(X_te_sub)
    accuracy = accuracy_score(y_te, preds)

    # Small penalty to favor fewer features if accuracies match (parsimony pressure)
```

```
    feature_penalty = 0.01 * (len(selected_indices) / NUM_FEATURES)
    fitness = accuracy - feature_penalty

    return max(0.0, fitness)

def selection_roulette(population, fitnesses):
    """Selects a parent using Roulette Wheel Selection."""
    total_fit = sum(fitnesses)
    if total_fit == 0:
        return population[np.random.randint(len(population))]
    probs = [f / total_fit for f in fitnesses]
    selected_idx = np.random.choice(len(population), p=probs)
    return population[selected_idx]

def crossover_single_point(parent1, parent2):
    """Combines two parents at a random split point to create two children."""
    if np.random.rand() < 0.8: # 80% crossover chance
        point = np.random.randint(1, NUM_FEATURES - 1)
        child1 = np.concatenate((parent1[:point], parent2[point:]))
        child2 = np.concatenate((parent2[:point], parent1[point:]))
    else:
        child1, child2 = parent1.copy(), parent2.copy()
    return child1, child2

def mutate(chromosome, rate):
    """Flips genes randomly based on the mutation rate."""
    for i in range(len(chromosome)):
        if np.random.rand() < rate:
            chromosome[i] = 1 - chromosome[i] # Flips 1 to 0 or 0 to 1
    # Check again to avoid an empty feature set
    if np.sum(chromosome) == 0:
        chromosome[np.random.randint(len(chromosome))] = 1
    return chromosome

# ------------------------------------------------------------
# 4. EXECUTION LOOP
# ------------------------------------------------------------
population = initialize_population(POP_SIZE, NUM_FEATURES)
best_overall_chromosome = None
best_overall_fitness = -1
history = []

print("\n--- Starting Genetic Algorithm Optimisation ---")

for gen in range(GENERATIONS):
    # 1. Calculate fitness for the whole population
    fitnesses = [calculate_fitness(chrom, X_train, X_test, y_train, y_test) for chrom in population]

    # Track the generation's best
    best_gen_idx = np.argmax(fitnesses)
    best_gen_fitness = fitnesses[best_gen_idx]
    history.append(best_gen_fitness)

    if best_gen_fitness > best_overall_fitness:
        best_overall_fitness = best_gen_fitness
        best_overall_chromosome = population[best_gen_idx].copy()

    print(f"Generation {gen+1:02d}: Best Fitness = {best_gen_fitness:.4f} | Features Kept = {np.sum(population[best_gen_idx])}")

    # 2. Build the next generation
    next_population = []
    # Elitism: Automatically carry over the best chromosome to save progress
    next_population.append(population[best_gen_idx])
    next_population.append(best_overall_chromosome)

    while len(next_population) < POP_SIZE:
        p1 = selection_roulette(population, fitnesses)
        p2 = selection_roulette(population, fitnesses)

        c1, c2 = crossover_single_point(p1, p2)

        next_population.append(mutate(c1, MUTATION_RATE))
        if len(next_population) < POP_SIZE:
            next_population.append(mutate(c2, MUTATION_RATE))

    population = np.array(next_population)

# ------------------------------------------------------------
```

```
# 5. FINAL RESULTS SUMMARY
# ------------------------------------------------------------
final_features = [feature_names[i] for i in range(NUM_FEATURES) if best_overall_chromosome[i] == 1]

print("\n================ FINAL RESULTS ================")
print(f"Optimal Number of Features: {len(final_features)} / {NUM_FEATURES}")
print(f"Selected Feature Set: {final_features}")
print(f"Best Fitness Achieved: {best_overall_fitness:.4f}")
```

```
Dataset initialized with 200 animals and 20 features.

--- Starting Genetic Algorithm Optimisation ---
Generation 01: Best Fitness = 0.7132 | Features Kept = 7
Generation 02: Best Fitness = 0.7460 | Features Kept = 8
Generation 03: Best Fitness = 0.7460 | Features Kept = 8
Generation 04: Best Fitness = 0.7460 | Features Kept = 8
Generation 05: Best Fitness = 0.7460 | Features Kept = 8
Generation 06: Best Fitness = 0.7460 | Features Kept = 8
Generation 07: Best Fitness = 0.7460 | Features Kept = 8
Generation 08: Best Fitness = 0.7960 | Features Kept = 8
Generation 09: Best Fitness = 0.7960 | Features Kept = 8
Generation 10: Best Fitness = 0.7960 | Features Kept = 8
Generation 11: Best Fitness = 0.7965 | Features Kept = 7
Generation 12: Best Fitness = 0.7965 | Features Kept = 7
Generation 13: Best Fitness = 0.7965 | Features Kept = 7
Generation 14: Best Fitness = 0.7965 | Features Kept = 7
Generation 15: Best Fitness = 0.7965 | Features Kept = 7

================ FINAL RESULTS ================
Optimal Number of Features: 7 / 20
Selected Feature Set: ['Ear_Shape', 'Meow_Vol', 'Tail_Length', 'Snout_Length', 'Fur_Thickness', 'Night_Vision', 'Sleep_Hours']
Best Fitness Achieved: 0.7965
```